

Solution for Martian Sieve

The first two subtasks show us special cases that are the two extremes between which the full solution lies: if using the sieve is too expensive, you just throw in all the rocks and use it once, and if handling a rock is too expensive, sieving boils down to binary search.

In order to get a polynomial-time solution, we'll going to need several observations.

Not using the biggest rock

We never need to use the rock of size n , we already know it will stay in the sieve. Hence, below we assume $a_i < n$.

States of the search

In the initial state there are n possible answers: s can be any of the values from 1 to n .

What information do we get by performing one round?

Let $a_1 < \dots < a_r$ be the r rocks we put into the sieve, and additionally let $a_0 = 0$ (a rock that definitely fell through) and $a_{r+1} = n$ (a rock that definitely stayed in the sieve). If the robot reports that k of the rocks fell through, this means that $a_k < s \leq a_{k+1}$. In other words, the new information is that s lies in the interval $[a_k + 1, a_{k+1}]$.

After any number of rounds, the total information we have about s is therefore simply the intersecion of all these intervals – which is still an interval.

Eliminating useless moves and normalizing states

Once we know that the answer is in some interval $[a, b]$, it doesn't make any sense to use rocks with sizes $< a$ or $\geq b$: we already know what would happen to them, so we would just be wasting time when adding them to the sieve.

Once we realize this, we can observe that there is no substantial difference between the problem "find s in the interval $[a, a + x - 1]$ " and the problem "find s in the interval $[1, x]$ ". In both cases we have exactly x possible answers left, and if we never make any useless moves, there is an obvious one-to-one mapping between optimal solutions for the two problems: using a rock of size i in the $[1, x]$ problem corresponds to using a rock of size $i + a - 1$ in the other one. Thus, the problems must have the same optimal time.

Hence, let T_x denote the optimal time in which we can find the correct s among any x consecutive options. Our goal is to determine T_n .

Optimal answers are non-decreasing

Clearly, any strategy that solves the problem for $x + 1$ options can also be used to solve the problem for x options (we just pretend that the next answer is also still possible and use the exact same strategy). Thus, T_{x+1} can never be smaller than T_x .

Analyzing a round again

Now that we know more about the new states, let's take a closer look at the effect of one round of sieving. We chose some number r of rocks and the specific sizes $a_1 < \dots < a_r$ of those rocks. (Remember that we are only using rocks for which we don't know whether they stay in the sieve.)

The robot will report the number of rocks that stayed in the sieve: some value k between 0 and r , inclusive. And once we know k , we are left with a new, smaller state. More precisely, the new number of options becomes $a_{k+1} - a_k$.

What will be the total time our solution will need to finish the search from this point if we continue optimally? As we just observed that t is non-decreasing, the worst case will clearly correspond to the largest number of options: the value $m = \max_k(a_{k+1} - a_k)$.

In total, we have spent $r \cdot c_r + c_s$ seconds to perform our first round and then we'll spend at most T_m additional seconds to finish the search.

Greedy selection of rocks

For a specific problem size n , once we fix r , we have fixed the duration of the round itself. In order to optimize the worst-case time, we want to minimize the value m .

By definition, m is the maximum of $k + 1$ options, and we can observe that the sum of those options is exactly n (either by evaluating a simple telescoping sum, or by simply observing that each of the n possible answers has to end up in one of the $k + 1$ branches).

Thus, m can never be smaller than $\lceil n/(k + 1) \rceil$. And we can actually reach this specific value m , e.g., by using rock sizes that are multiples of the desired m . (If n is not an exact multiple of $k + 1$, the last segment will be shorter.)

Dynamic programming

At this point we know enough to solve the problem for moderate sizes of n . For each i from 1 to n we will calculate the value T_i as follows:

- We'll iterate over all possible values of r .
- For each r , we will greedily pick the best query and calculate the worst-case time it produces.
- We'll take the best (i.e., smallest) among those options.

This gives us a solution in $O(n^2)$ time.

What remains is to solve the last subproblem. The solutions so far have been generic, the solution to the last subproblem will depend on the fact that c_r and c_s are small integers.

General idea of the full solution

Below, we will show that the optimal time will be roughly logarithmic in n and that in the optimal solution the number of rocks used in each round will be reasonably small. This will allow us to actually compute all the values T_1 to T_n without having to evaluate each individual T_i .

Limiting the round size

As an example, suppose you want to make a round with $r = 15$ rocks.

Consider a second option: make a round with just three rocks (namely, a_4 , a_8 , and a_{12}) and then based on the answer you get a second round, again with only three new rocks. E.g., if the first round tells you that the answer is between a_4 and a_8 , the second round will use rocks a_5 , a_6 , and a_7 .

The information we'll have at the end is exactly the same in both cases. Hence, whenever the second option happens to be faster than the first, we should prefer it.

The second option pays the constant overhead c_s twice instead of once, but this gets balanced by spending less time on individual rocks: we go down from $15c_r$ to just $6c_r$.

Asymptotically, we are going from r rocks to roughly $2\sqrt{r}$ rocks, so the new time becomes roughly $2\sqrt{r}c_r + 2c_s$. Thus, the second option is faster than the first whenever $c_s < c_r(r - 2\sqrt{r})$. As the values c_s and c_r are fixed constants, it is always the case that once r becomes large enough, the first option becomes slower than the second one. Thus, there is always an optimal solution in which r is never large.

For our constraints, we can convert the above rough argument into an exact one.

Theorem: If $c_s \leq 100$, there is always an optimal solution such that in each round the number of r rocks used is at most 120.

Proof: Suppose $r \geq 121$. If we instead use 11 equally spaced rocks in the first round, this will divide the remaining $r - 11$ rocks into 12 roughly equal groups. In the second round we will use only one of those 12 groups. We can easily verify that this will leave at least 100 unused rocks in each case. Thus, we will save at least $100c_r \geq 100$ seconds while paying an extra $c_s \leq 100$ seconds for the extra round, and therefore the new solution is at least as fast as the old one.

Full solution

We always have the option of just binary searching the solution. This will require about $\log_2 n$ rounds, each taking $c_r + c_s$ seconds. This gives a rather small upper bound on the optimal answer. For our maximal constraints, the answer will never exceed 10 800.

In combination with the observation that the answers are non-decreasing this means that the whole range from 1 to n can be divided into an at-most-logarithmic number of blocks such that within each block the answer is constant.

In our full solution we will compute these blocks explicitly.

Suppose the last computed block ends with x . We can explicitly compute T_{x+1} to get the answer that corresponds to the next block, and then use binary search to find the largest y such that $T_y = T_{x+1}$. Each individual T_i is calculated the same way as in the quadratic solution, with two improvements:

- we only iterate over small values of r
- if y is so large that after first round we can still be outside the computed blocks, then clearly y is too large to be in the next block

An alternative full solution

Instead of computing, for each number of options x , the optimal time T_x , we can turn the dynamic programming around: index it by time and compute how many options that time buys us.

Let D_t be the largest number of consecutive options among which we can always pin down the answer in at most t seconds in total. With no time to spare we can only handle a single option, so $D_t = 1$ for every t below the cost $c_r + c_s$ of the cheapest possible round. For larger t we again consider one round using r rocks. Such a round costs $r c_r + c_s$ seconds and splits the options into $r + 1$ groups. In the worst case the answer lies in the largest group, and from there we still have $t - (r c_r + c_s)$ seconds left, so each group may contain up to $D_{t-(r c_r + c_s)}$ options. This gives

$$D_t = \max \left(1, \max_{r \geq 1, r c_r + c_s \leq t} (r + 1) D_{t-(r c_r + c_s)} \right).$$

The optimal time for the actual problem is then the smallest t for which $D_t \geq n$.

Two remarks make this efficient. First, we never need t larger than the binary-search upper bound derived above (never more than about 10 800 for our constraints), and by the same round-size theorem we only iterate over small values of r ; the whole DP therefore runs in time independent of n . Second, D_t grows multiplicatively and would overflow quickly, so we cap it at n . This solves the task without ever explicitly maintaining the blocks of constant answer.